

Series de Tiempo:

Modelado, Análisis y Pronóstico

Documentación de Notebooks de Python

Iñaki Castillo

Junio de 2026

Índice

1. Introducción	2
2. Caminata Aleatoria (<i>Random Walk</i>)	3
2.1. Descripción del Notebook	3
2.2. Fundamentos Teóricos	3
2.2.1. Diferenciación	3
2.2.2. Test de Dickey-Fuller Aumentado (ADF)	4
2.3. Código: Simulación	4
2.4. Código: Diferenciación y Test ADF	4
3. Series Financieras — Amazon (AMZN)	6
3.1. Descripción del Notebook	6
3.2. Fundamentos Teóricos	6
3.2.1. Precios de cierre ajustados	6
3.2.2. Log-retornos	6
3.3. Código	6
4. Estacionalidad y SARIMAX	8
4.1. Descripción del Notebook	8
4.2. Fundamentos Teóricos	8
4.2.1. Descomposición de una serie temporal	8
4.2.2. Modelo SARIMAX	8
4.3. Generación del Dataset Sintético	8
5. Modelo SARIMA y Prophet	11
5.1. Descripción del Notebook	11
5.2. Fundamentos Teóricos	11
5.2.1. Modelo SARIMA(0, 1, 1)(0, 1, 1) ₁₂	11
5.2.2. Diagnóstico de Residuos	11
5.2.3. Modelo Prophet	12
5.3. Código: Generación de Datos y Ajuste SARIMA	12
5.4. Código: Pronóstico SARIMA	13
5.5. Código: Análisis de Residuos	13
5.6. Código: Pronóstico con Prophet	14
6. Comparación de Modelos	16
7. Dependencias y Entorno	17
Referencias	17

1. Introducción

Este documento recopila y documenta cuatro notebooks de Python orientados al análisis de series de tiempo. Los temas cubiertos van desde los fundamentos teóricos —como el proceso de caminata aleatoria— hasta técnicas de modelado más avanzadas como SARIMA, SARIMAX y Prophet, pasando por el análisis de series financieras reales obtenidas con `yfinance`.

La notación general que se utiliza a lo largo del documento es la siguiente:

Símbolo	Significado
$\{X_t\}$	Proceso estocástico (serie de tiempo)
Z_t	Ruido blanco gaussiano: $Z_t \sim \mathcal{N}(0, \sigma^2)$
Δ	Operador de diferencia: $\Delta X_t = X_t - X_{t-1}$
B	Operador de retroceso (<i>backshift</i>): $BX_t = X_{t-1}$
p, d, q	Órdenes AR, integración y MA (no estacional)
P, D, Q	Órdenes AR, integración y MA estacional
m	Periodo estacional

2. Caminata Aleatoria (*Random Walk*)

2.1. Descripción del Notebook

El notebook `caminata_aleat.ipynb` introduce el concepto de **caminata aleatoria**, uno de los procesos estocásticos más importantes en el análisis de series de tiempo y en finanzas. Se simula el proceso, se diferencia para obtener una serie estacionaria y se aplica el test de Dickey-Fuller Aumentado (ADF) para confirmar la estacionariedad.

2.2. Fundamentos Teóricos

Definición

Una **caminata aleatoria** es un proceso estocástico $\{X_t\}$ definido por la recurrencia:

$$X_t = X_{t-1} + Z_t, \quad t = 1, 2, \dots, T$$

donde $Z_t \stackrel{iid}{\sim} \mathcal{N}(0, 1)$ es ruido blanco gaussiano y X_0 es un valor inicial fijo (generalmente $X_0 = 0$).

La solución explícita del proceso es:

$$X_t = X_0 + \sum_{k=1}^t Z_k$$

Sus propiedades estadísticas son:

$$\begin{aligned} \mathbb{E}[X_t] &= X_0 = 0 \\ \text{Var}[X_t] &= t\sigma^2 \end{aligned}$$

Como la varianza crece linealmente con el tiempo, la caminata aleatoria **no es estacionaria en covarianza**. Esto tiene implicaciones directas para el modelado: aplicar modelos ARMA directamente a este proceso sería incorrecto.

2.2.1. Diferenciación

Para convertir el proceso en estacionario se aplica el **operador de diferencia**:

$$\Delta X_t = X_t - X_{t-1} = Z_t$$

La serie diferenciada $\{\Delta X_t\}$ es ruido blanco con media cero y varianza constante σ^2 , es decir, *sí* es estacionaria.

2.2.2. Test de Dickey-Fuller Aumentado (ADF)

El ADF contrasta las hipótesis:

H_0 : la serie tiene raíz unitaria (no es estacionaria) H_1 : la serie es estacionaria

Se rechaza H_0 cuando el estadístico ADF es **más negativo** que el valor crítico, o equivalentemente cuando el $p\text{-valor} < 0,05$.

2.3. Código: Simulación

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Par metros
5 np.random.seed(42)      # Reproducibilidad
6 T = 200                  # Numero de periodos
7 X0 = 0                   # Valor inicial
8
9 # Generar ruido blanco  $Z_t \sim N(0,1)$ 
10 Z = np.random.normal(0, 1, T)
11
12 # Generar proceso  $X_t = X_{t-1} + Z_t$ 
13 X = np.zeros(T)
14 X[0] = X0
15
16 for t in range(1, T):
17     X[t] = X[t-1] + Z[t]
18
19 # Gráfica
20 plt.figure(figsize=(12, 6), dpi=120)
21 plt.plot(X, linewidth=2.5)
22 plt.scatter(range(T), X, s=15)
23 plt.title("Simulación de Random Walk\n $X_t = X_{t-1} + Z_t$ ", fontsize=16)
24 plt.xlabel("Tiempo (t)", fontsize=12)
25 plt.ylabel("Valor de  $X_t$ ", fontsize=12)
26 plt.grid(alpha=0.3)
27 plt.axhline(0, linestyle='--')
28 plt.tight_layout()
29 plt.show()

```

Listing 1: Simulación de la caminata aleatoria

2.4. Código: Diferenciación y Test ADF

```

1 from statsmodels.tsa.stattools import adfuller
2
3 # Diferenciación: Delta  $X_t = X_t - X_{t-1}$ 
4 X_diff = np.diff(X)
5
6 # Gráfica de la serie diferenciada
7 plt.figure(figsize=(12, 6), dpi=120)
8 plt.plot(X_diff, linewidth=2.5)
9 plt.scatter(range(len(X_diff)), X_diff, s=15)
10 plt.title("Serie Diferenciada\n $\Delta X_t = X_t - X_{t-1}$ ", fontsize=16)
11 plt.xlabel("Tiempo (t)", fontsize=12)
12 plt.ylabel("Valor Diferenciado", fontsize=12)
13 plt.grid(alpha=0.3)
14 plt.axhline(0, linestyle='--')
15 plt.tight_layout()
16 plt.show()
17
18 # Test ADF
19 adf_result = adfuller(X_diff)
20
21 print(f"Estadístico ADF : {adf_result[0]:.4f}")
22 print(f"p-valor : {adf_result[1]:.4f}")
23 print("Valores críticos:")
24 for key, value in adf_result[4].items():
25     print(f"    {key}: {value:.4f}")
26
27 if adf_result[1] < 0.05:
28     print("\nConclusión: La serie diferenciada ES
29         estacionaria (rechazamos H0).")
30 else:
31     print("\nConclusión: La serie diferenciada NO es
32         estacionaria.")

```

Listing 2: Diferenciación y Test ADF sobre la caminata aleatoria

Resultado clave

La serie $\Delta X_t = Z_t$ es ruido blanco, por lo que el test ADF rechaza la hipótesis nula de raíz unitaria. Se espera un estadístico muy negativo y un p -valor $\ll 0,05$, confirmando que diferenciar **una sola vez** es suficiente para hacer estacionaria a la caminata aleatoria.

3. Series Financieras — Amazon (AMZN)

3.1. Descripción del Notebook

El notebook `Series_Financieras.ipynb` descarga precios históricos de la acción de Amazon (ticker: AMZN) mediante la librería `yfinance`, visualiza la serie de precios de cierre con `Altair` y calcula los **log-retornos** como transformación estándar en finanzas cuantitativas.

3.2. Fundamentos Teóricos

3.2.1. Precios de cierre ajustados

En finanzas cuantitativas se trabaja habitualmente con el precio de cierre ajustado P_t que incorpora dividendos y *splits*. El precio crudo es no estacionario (generalmente con tendencia estocástica), por lo que no puede emplearse directamente en modelos ARMA sin transformación.

3.2.2. Log-retornos

Sea P_t el precio de cierre en el periodo t . El **log-retorno continuo** se define como:

$$r_t = \ln\left(\frac{P_t}{P_{t-1}}\right) = \ln P_t - \ln P_{t-1}$$

Propiedades importantes de los log-retornos:

- Son **aditivos en el tiempo**: el retorno acumulado en $[1, T]$ es $\sum_{t=1}^T r_t$.
- Para retornos pequeños, $r_t \approx (P_t - P_{t-1})/P_{t-1}$ (retorno aritmético).
- Empíricamente presentan *colas pesadas y agrupamiento de volatilidad (volatility clustering)*.
- Son aproximadamente estacionarios en media (aunque su varianza puede variar, capturada por modelos ARCH/GARCH).

3.3. Código

```
1 import yfinance as yf
2 import pandas as pd
3 import numpy as np
4 import altair as alt
5
6 # Descarga de datos
7
7 amzn = yf.download("AMZN", start="2010-01-01")
8 amzn_precios = amzn["Close"]
9
```

```

10 # Visualización interactiva con Altair
11 df = amzn_precios.reset_index()
12 df.columns = ["Date", "Close"]
13
14 chart = (
15     alt.Chart(df)
16     .mark_line()
17     .encode(
18         x=alt.X("Date:T", title="Fecha"),
19         y=alt.Y("Close:Q", title="Precio de cierre (USD)"),
20         tooltip=["Date:T", "Close:Q"],
21     )
22     .properties(title="Serie de tiempo AMZN", width=900, height=400)
23     .interactive()
24 )
25 chart
26
27 # Log-retornos
28 #  $r_t = \ln(P_t) - \ln(P_{t-1})$ 
29 amzn_log_returns = np.log(amzn_precios /
30     amzn_precios.shift(1))
31 amzn_log_returns = amzn_log_returns.dropna() # Eliminar primer NaN
32 display(amzn_log_returns.head())

```

Listing 3: Descarga de datos y cálculo de log-retornos de AMZN

Nota

El primer valor del vector de log-retornos resulta NaN porque no existe P_{t-1} para $t = 0$. Se elimina con `.dropna()` antes de cualquier análisis posterior. En la práctica, los log-retornos de AMZN desde 2010 presentan el comportamiento típico de renta variable: media cercana a cero, exceso de curtosis (colas pesadas) y períodos de alta y baja volatilidad.

4. Estacionalidad y SARIMAX

4.1. Descripción del Notebook

El notebook `Seasonality.ipynb` construye un dataset sintético con **tendencia**, **estacionalidad anual** y una **variable exógena** (efecto de promociones) para motivar el uso del modelo SARIMAX, que es la extensión de SARIMA con variables explicativas externas.

4.2. Fundamentos Teóricos

4.2.1. Descomposición de una serie temporal

Una serie de tiempo $\{Y_t\}$ puede descomponerse como:

$$Y_t = T_t + S_t + \varepsilon_t$$

donde T_t es la **tendencia**, S_t es el componente **estacional** y ε_t es el término de **error**.

4.2.2. Modelo SARIMAX

El modelo SARIMAX $(p, d, q)(P, D, Q)_m$ extiende SARIMA incluyendo regresores exógenos $\mathbf{x}_t$:

$$\Phi_P(B^m) \phi_p(B) \Delta^d \Delta_m^D Y_t = \Theta_Q(B^m) \theta_q(B) \varepsilon_t + \beta^\top \mathbf{x}_t$$

donde:

$\phi_p(B) = 1 - \phi_1 B - \dots - \phi_p B^p$	(polinomio AR no estacional)
$\theta_q(B) = 1 + \theta_1 B + \dots + \theta_q B^q$	(polinomio MA no estacional)
$\Phi_P(B^m) = 1 - \Phi_1 B^m - \dots - \Phi_P B^{Pm}$	(polinomio AR estacional)
$\Theta_Q(B^m) = 1 + \Theta_1 B^m + \dots + \Theta_Q B^{Qm}$	(polinomio MA estacional)
$\Delta^d = (1 - B)^d$	(diferenciación no estacional)
$\Delta_m^D = (1 - B^m)^D$	(diferenciación estacional)

4.3. Generación del Dataset Sintético

```

1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 # Rango de fechas: 5 a os, datos mensuales

```

```

6  dates = pd.date_range(start='2018-01-01', periods=5*12,
7      freq='MS')
8
9  # Componentes de la serie
10 trend      = np.linspace(0, 100, len(dates))
11                                     # Tendencia lineal
12 seasonality = (15 * np.sin(np.linspace(0, 3*2*np.pi,
13     len(dates)))
14                                     # Ciclo anual
15     + 10 * np.cos(np.linspace(0, 2*np.pi,
16     len(dates))))
17 noise      = np.random.normal(0, 5, len(dates))
18
19 # Variable ex gena: efecto de promociones
20 promotion_effect = np.zeros(len(dates))
21 promotion_months = [3, 8, 15, 20, 27, 32, 39, 44, 51, 56]
22
23 for month_idx in promotion_months:
24     if month_idx < len(dates):
25         promotion_effect[month_idx] = 20 +
26             np.random.normal(0, 3)
27
28 # Serie final: combinaci n aditiva
29 time_series = trend + seasonality + promotion_effect +
30     noise
31
32 # DataFrame
33 df = pd.DataFrame(
34     {'value': time_series, 'promotion': promotion_effect
35     > 0},
36     index=dates
37 )
38 display(df.head())
39
40 # Visualizaci n
41 plt.figure(figsize=(15, 7))
42 plt.plot(df.index, df['value'], label='Serie Temporal
43     Sint tica')
44 plt.scatter(df.index[df['promotion']],
45     df['value'][df['promotion']],
46     color='red', s=50, label='Mes con Promoci n')
47 plt.title('Serie Temporal Sint tica con Tendencia,
48     Estacionalidad y Promociones')
49 plt.xlabel('Fecha')
50 plt.ylabel('Valor')

```

```
41 plt.legend()  
42 plt.grid(True)  
43 plt.show()
```

Listing 4: Construcción del dataset sintético con tendencia, estacionalidad y variables exógenas

Nota

En escenarios reales, la columna `promotion` puede reemplazarse por cualquier variable exógena relevante: días festivos, índices económicos, temperatura, campañas de marketing, etc. SARIMAX aprovecha esta información para mejorar la precisión del pronóstico en comparación con un SARIMA puro.

5. Modelo SARIMA y Prophet

5.1. Descripción del Notebook

El notebook `Clase_1.ipynb` implementa paso a paso:

1. Generación de una serie de tiempo sintética con tendencia y estacionalidad.
2. Ajuste de un modelo **SARIMA**(0, 1, 1)(0, 1, 1)₁₂ (Airline Model).
3. Pronóstico a 24 meses.
4. Análisis de residuos (ACF, PACF, Q-Q plot, tests estadísticos).
5. Pronóstico alternativo con el modelo **Prophet** de Meta.

5.2. Fundamentos Teóricos

5.2.1. Modelo SARIMA(0, 1, 1)(0, 1, 1)₁₂

Conocido como el *Airline Model* (Box & Jenkins, 1976), es uno de los modelos de series de tiempo más utilizados en la práctica. Su ecuación es:

$$(1 - B)(1 - B^{12})Y_t = (1 + \theta_1 B)(1 + \Theta_1 B^{12})\varepsilon_t$$

Las componentes de este modelo específico son:

Parámetro	Valor	Interpretación
p	0	Sin AR no estacional
d	1	Una diferenciación no estacional
q	1	MA de orden 1 no estacional
P	0	Sin AR estacional
D	1	Una diferenciación estacional
Q	1	MA de orden 1 estacional
m	12	Periodo anual (datos mensuales)

El doble operador de diferencia $\Delta\Delta_{12}$ elimina simultáneamente la **tendencia** y la **estacionalidad** de la serie.

5.2.2. Diagnóstico de Residuos

Para validar el ajuste del modelo los residuos $\hat{\varepsilon}_t$ deben comportarse como **ruido blanco**:

- **Ljung-Box**: contrasta H_0 : no hay autocorrelación en los residuos. $p > 0,05$ es deseable.

- **Jarque-Bera:** contrasta H_0 : los residuos son normales. $p > 0,05$ es deseable.
- **Heteroscedasticidad:** contrasta H_0 : varianza constante. $p > 0,05$ es deseable.

5.2.3. Modelo Prophet

Prophet es un modelo de pronóstico aditivo desarrollado por Meta (2017):

$$Y(t) = g(t) + s(t) + h(t) + \varepsilon_t$$

donde $g(t)$ es la tendencia, $s(t)$ los componentes estacionales (Fourier), $h(t)$ el efecto de días festivos y ε_t el error. Es especialmente robusto ante datos faltantes y cambios de tendencia.

5.3. Código: Generación de Datos y Ajuste SARIMA

```

1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 np.random.seed(42)
6 n_points = 120 # 10 a os de datos mensuales
7 index = pd.date_range(start='2010-01-01',
8                       periods=n_points, freq='MS')
9
10 # Componentes
11 trend = np.linspace(0, 20, n_points)
12 seasonal = 10 * np.sin(np.linspace(0, 3 * np.pi,
13                                   n_points) * 2)
14 noise = np.random.normal(0, 1.5, n_points)
15
16 # Serie resultante
17 data = trend + seasonal + noise + 50
18 time_series = pd.Series(data, index=index)
19
20 plt.figure(figsize=(12, 6))
21 plt.plot(time_series)
22 plt.title('Serie de Tiempo Sintética')
23 plt.xlabel('Fecha')
24 plt.ylabel('Valor')
25 plt.grid(True)
26 plt.show()

```

Listing 5: Generación de la serie sintética

```

1 from statsmodels.tsa.statespace.sarimax import SARIMAX

```

```

2
3 order          = (0, 1, 1)
4 seasonal_order = (0, 1, 1, 12)
5
6 model          = SARIMAX(time_series, order=order,
7                          seasonal_order=seasonal_order,
8                          enforce_stationarity=False,
9                          enforce_invertibility=False)
10 model_fit = model.fit(dispatch=False)
11
12 print(model_fit.summary())

```

Listing 6: Ajuste del modelo SARIMA(0, 1, 1)(0, 1, 1)₁₂

5.4. Código: Pronóstico SARIMA

```

1 n_forecast = 24
2
3 forecast = model_fit.predict(
4     start=len(time_series),
5     end=len(time_series) + n_forecast - 1
6 )
7
8 forecast_index = pd.date_range(
9     start=time_series.index[-1] + pd.DateOffset(months=1),
10    periods=n_forecast, freq='MS'
11 )
12 forecast.index = forecast_index
13
14 plt.figure(figsize=(14, 7))
15 plt.plot(time_series, label='Datos Originales')
16 plt.plot(forecast, label='Pronóstico SARIMA',
17          color='red', linestyle='--')
18 plt.title('Serie Sintética con Pronóstico SARIMA')
19 plt.xlabel('Fecha')
20 plt.ylabel('Valor')
21 plt.legend()
22 plt.grid(True)
23 plt.show()

```

Listing 7: Pronóstico a 24 meses con SARIMA

5.5. Código: Análisis de Residuos

```

1 import seaborn as sns

```

```

2 import statsmodels.api as sm
3 from statsmodels.graphics.tsaplots import plot_acf,
  plot_pacf
4
5 residuals = model_fit.resid
6
7 #           Gráficas de diagnóstico
8
9 fig, axes = plt.subplots(3, 1, figsize=(16, 10))
10
11 axes[0].plot(residuals)
12 axes[0].set_title('Residuos a lo Largo del Tiempo')
13 axes[0].grid(True)
14
15 sns.histplot(residuals, kde=True, ax=axes[1])
16 axes[1].set_title('Distribución de los Residuos')
17
18 sm.qqplot(residuals, line='s', ax=axes[2])
19 axes[2].set_title('Q-Q Plot de los Residuos')
20
21 plt.tight_layout()
22 plt.show()
23
24 #           ACF y PACF
25
26 fig2, (ax1, ax2) = plt.subplots(2, 1, figsize=(14, 8))
27 plot_acf(residuals, lags=40, ax=ax1, zero=False)
28 plot_pacf(residuals, lags=40, ax=ax2, zero=False)
29 plt.tight_layout()
30 plt.show()

```

Listing 8: Diagnóstico gráfico y estadístico de los residuos

Resultado clave

Para el modelo ajustado se esperan los siguientes valores en los tests del `summary()`: Ljung-Box $p \approx 0,76$, Jarque-Bera $p \approx 0,53$, Heteroscedasticidad $p \approx 0,64$. Todos son $> 0,05$, lo que indica que los residuos se comportan como ruido blanco y el modelo es adecuado.

5.6. Código: Pronóstico con Prophet

```

1 from prophet import Prophet
2
3 # Prophet requiere columnas 'ds' (fecha) e 'y' (valor)
4 prophet_df = time_series.reset_index()

```

```
5 prophet_df.columns = ['ds', 'y']
6
7 # Ajuste del modelo
8 model_prophet = Prophet(yearly_seasonality=True)
9 model_prophet.fit(prophet_df)
10
11 # DataFrame de fechas futuras
12 future =
13     model_prophet.make_future_dataframe(periods=n_forecast,
14     freq='MS')
15 forecast_prophet = model_prophet.predict(future)
16
17 # Columnas principales del pronóstico
18 display(forecast_prophet[['ds', 'yhat', 'yhat_lower',
19     'yhat_upper']].tail())
20
21 # Visualización
22 fig = model_prophet.plot(forecast_prophet)
23 plt.title('Serie Sintética con Pronóstico de Prophet')
24 plt.xlabel('Fecha')
25 plt.ylabel('Valor')
26 plt.show()
27
28 # Descomposición de componentes
29 fig2 = model_prophet.plot_components(forecast_prophet)
30 plt.show()
```

Listing 9: Ajuste y pronóstico con el modelo Prophet

Nota

Prophet descompone automáticamente el pronóstico en tendencia y estacionalidad. Las columnas `yhat_lower` y `yhat_upper` representan el intervalo de credibilidad al 80 % por defecto. A diferencia de SARIMA, Prophet no requiere que la serie sea estacionaria ni que el usuario especifique p , d , q .

6. Comparación de Modelos

Característica	SARIMA	SARIMAX	Prophet
Variables exógenas	No	Sí	Días festivos
Estacionalidad múltiple	No	No	Sí
Requiere estacionariedad	Sí	Sí	No
Selección de hiperparámetros	Manual	Manual	Automática
Intervalo de predicción	Frecuentista	Frecuentista	Bayesiano
Robusto a datos faltantes	No	No	Sí

7. Dependencias y Entorno

Las siguientes librerías de Python son necesarias para reproducir todos los notebooks:

```
1 pip install numpy pandas matplotlib seaborn statsmodels
   yfinance altair prophet
```

Listing 10: Instalación de dependencias

Librería	Uso principal	Versión recomendada
numpy	Álgebra lineal y generación de datos	$\geq 1,24$
pandas	Manejo de series temporales	$\geq 2,0$
matplotlib	Visualización estática	$\geq 3,7$
seaborn	Visualización estadística	$\geq 0,12$
statsmodels	SARIMA, ADF, ACF, PACF	$\geq 0,14$
yfinance	Descarga de datos financieros	$\geq 0,2$
altair	Gráficas interactivas	$\geq 5,0$
prophet	Modelo Prophet (Meta)	$\geq 1,1$

Referencias

- [1] Box, G.E.P., Jenkins, G.M., Reinsel, G.C. & Ljung, G.M. (2015). *Time Series Analysis: Forecasting and Control*. 5th ed. Wiley.
- [2] Hyndman, R.J. & Athanasopoulos, G. (2021). *Forecasting: Principles and Practice*. 3rd ed. OTexts. <https://otexts.com/fpp3/>
- [3] Taylor, S.J. & Letham, B. (2018). Forecasting at Scale. *The American Statistician*, 72(1), 37–45.
- [4] Seabold, S. & Perktold, J. (2010). Statsmodels: Econometric and Statistical Modeling with Python. *9th Python in Science Conf*.